

CHAPTER 14

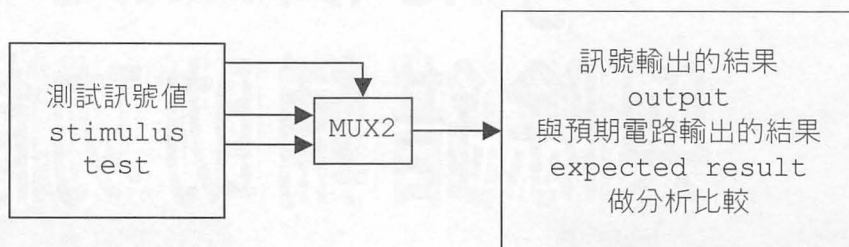
Verilog 的檔案處理 與除錯輔助功能

Verilog

在本章將為各位介紹：測試平台 (TestBench)、Verilog 的檔案處理以及除錯輔助功能。另外：本章中所介紹的語法是不能合成的，請特別留意。

14.1 測試平台 (TestBench)

測試平台 (TestBench) 是指使用 Verilog 的一些敘述去產生用來測試您所設計好的 module 所需要的測試訊號值 (stimulus) 以及時序 (timing)，然後再觀察該 module 訊號輸出的結果 (output) 與您所預期電路輸出的結果 (expected) 是否吻合，做分析比較這整個測試環境的通稱。



值得注意的是用於測試平台 (TestBench) 的 Verilog 敘述，像是對於檔案的處理以及訊號延遲…等等是不能用於電路的“合成”，只能用於電路的“模擬”。

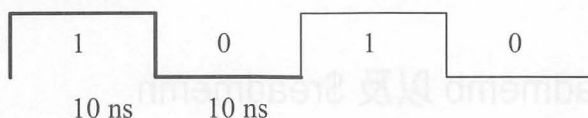
也就是說 FPGA 廠商所提供的電路“合成”軟體，只能處理可以用於電路“合成”的 Verilog 敘述；電路“模擬”軟體才有提供那些可以用於電路“模擬”的 Verilog 敘述。

例如：在測試平台 (TestBench) 中，您可以用 readmem、readmem、fdisplay、fmonitor…之類不能用於電路的“合成”的 Verilog 敘述。

例如，產生一個 Clk 訊號：

```
reg Clk;
initial
begin
    Clk = 1'b1;
end

always
begin
    #20 Clk = ~Clk; // 50 MHz
end
```



▲ 每 20 ns 產生一個 Clk 脈波訊號



14.2 Verilog 的檔案處理

14.2.1 \$fopen 以及 \$fclose

\$fopen 用於產生一個空的文字檔，配合 \$fdisplay、\$fwrite 以及 \$fstrobe 這一系列的 task，將電路模擬過程中產生的模擬結果或訊息輸出到檔案中。

理論上最多可以使用 32 個 \$fopen 來打開 32 個檔案，實際上可能沒那麼多，由作業系統而定。

\$fclose 通常用於電路模擬結束時，將 \$fopen 產生的文字檔關閉。

【語法】

```
integer file_handle; // 檔案代碼是 32 個位元的無號數整數
file_handle = $fopen( "<file_name>" );
$fclose( file_handle );
```

\$fopen 傳回來的值(檔案代碼)是 32 個位元的無號數整數，如果傳回值是 0 表示無法打開要用於模擬結果或訊息輸出的檔案。

檔案代碼的位元 0 等於 '1'，表示 standard output(標準輸出)，也就是會輸出到目前所打開用於輸出的每個檔案。

檔案代碼的位元 1 等於 '1'，表示第 1 個被打開的檔案；位元 2 等於 '1'，表示第 2 個被打開的檔案；...；位元 31 等於 '1'，表示第 31 個被打開的檔案。

14.2.2 \$readmemb 以及 \$readmemh

\$readmemb 以及 \$readmemh 這 2 個 task 用於從文字檔案讀取資料到 reg [3:0] Test_Pattern[1:16]；這種寫法的記憶體中，用於預設記憶體的內容。

\$readmemb 可以將文字檔案裡頭所放的二進制值(Binary format)讀取資料到記憶體中。

\$readmemh 可以將文字檔案裡頭所放的二進制值(Hexadecimal format)讀取資料到記憶體中。

【語法】

```
$readmemb( "File", MemoryName[, StartAddr[, FinishAddr]] );  
$readmemh( "File", MemoryName[, StartAddr[, FinishAddr]] );
```

其中："File" 是文字檔案的名稱，MemoryName 記憶體的名字，Start-Addr 記憶體開始的位址(可以不給定)，FinishAddr 記憶體結束的位址(可以不給定)。

文字檔案裡頭，每一筆資料之間的間隔可以是空白字元、Tab 或者是換行字元。

文字檔案裡頭可以給 @ 記憶體位址，表示接下來的資料要開始放到這個記憶體位址的後面，而記憶體位址一定要是 16 進制的值。

【例如】

```
@5  
1 a d
```

表示資料 1 會放在記憶體 5 的位址，a 會放在記憶體 6 的位址，d 會放在記憶體 7 的位址。

```
@1d  
b 2 6
```

表示資料 b 會放在記憶體 1d(也就是 29)的位址，2 會放在記憶體 1e 的位址，6 會放在記憶體 1f 的位址。

如果有給 StartAddr，那麼檔案裡頭的第 1 筆資料預設會被讀到 Start-Addr 的位址，下一個位址放第 2 筆資料，一直放到檔案裡頭的最後一筆資料或者是 FinishAddr 所指定的那個位址。

如果有給 StartAddr，那麼 StartAddr 必須 $\leq$ 檔案裡頭 @ 之後給的記憶體位址。

範例 test_readmemb.txt

```
@4  
  
0000  
0001  
0010  
0011  
  
0100  
0101
```

```

0110
0111

1000
1001
1010
1011

1100
1101
1110
1111

// readmem.TF
...
parameter Pattern_Number = 16;
reg [3:0] Test_Pattern[1:Pattern_Number];

initial
begin : Pattern_Read
    $readmemb( "test_readmemb.txt", Test_Pattern, 3, 6 );
end

```

這樣的話，結果：0000 會被讀到 4 的位址，0001 會被讀到 5 的位址，0010 會被讀到 6 的位址。

```

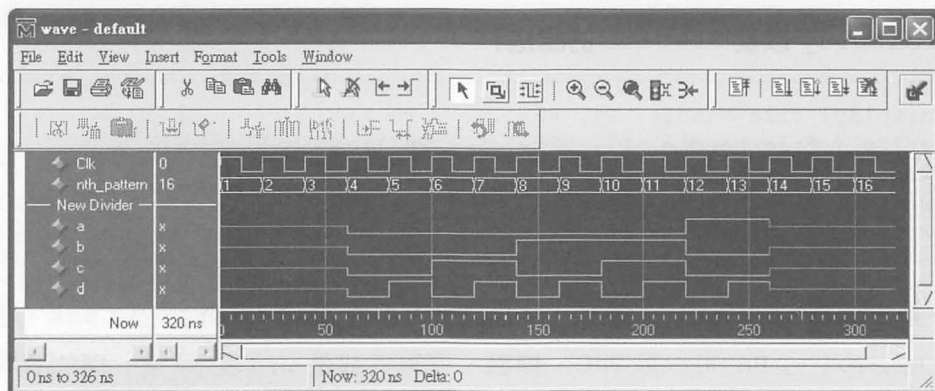
initial
begin : Pattern_Apply
    integer i;
    for (i = 1; i <= Pattern_Number; i = i + 1)
    begin
        {a, b, c, d} = Test_Pattern[i];

        $display("%8d: %b %2b %3b %4d", nth_pattern, a, b, c, d);

        #20
        ;
    end

    $finish;
end
...

```



▲ readmem.TF 的模擬結果

14.2.3 \$display、\$write、\$fdisplay 以及 \$fwrite

\$display 和 \$write 這二大類的 task 用於將模擬結果或訊息輸出到畫面上。

\$display 和 \$write 的功能大致相同，不同點在於 \$display 這一系列的 task 輸出到最後會自動換行，而 \$write 這一系列的 task 不會自動換行。

\$fdisplay 和 \$fwrite 這二大類的 task 用於將模擬結果或訊息輸出到檔案中。

\$fdisplay 和 \$fwrite 的功能大致相同，不同點在於 \$fdisplay 這一系列的 task 輸出到最後會自動換行，而 \$fwrite 這一系列的 task 不會自動換行。

【\$display 以及 \$write 的語法】

```
$display( P1, P2, ... , Pn );
$write ( P1, P2, ... , Pn );
$writeb( P1, P2, ... , Pn );
$writeo( P1, P2, ... , Pn );
$writeh( P1, P2, ... , Pn );
```


【\$fdisplay 以及 \$fwrite 的語法】

```
$fdisplay( file_handle, P1, P2, ... , Pn );  
$fwrite ( file_handle, P1, P2, ... , Pn );  
$fwriteb( file_handle, P1, P2, ... , Pn );  
$fwriteo( file_handle, P1, P2, ... , Pn );  
$fwriteh( file_handle, P1, P2, ... , Pn );
```

其中：file_handle 檔案代碼，請參考前面的 \$fopen 這個 task。參數 P1, P2, ... , Pn 可以是字串、變數、運算式或是 , (表示) 這一欄的資料留空。字串可以：escaped characters 或是 format specifiers。

escaped characters 可以是：

```
\n    Newline 換行  
\t    Tab 跳格  
\"    Double quote 雙引號  
\\    Back slash 反斜線  
\nnn 表示一個 8 進制的值
```

format specifiers 可以是：

```
%b %B Binary 二進制值  
%o %O Octal 八進制值  
%d %D Decimal 十進制值  
%h %H Hexadecimal 十六進制值  
%e %E %f %F %g %G Real 實數  
%c %C Character 字元  
%s %S String 字串  
%t %T Time 時間  
%m %M Hierarchical Instance 階層式名稱  
%v %V Binary and Strength 二進制值與訊號強度  
%v %V 輸出的值所代表的意義是～  
Su: supply, St: strong, Pu: Pull, La: Large,  
We: Weak, Me: Medium, Sm: Small, Hi: Highz,  
H: '1', L: '0', X: Unknown
```

format specifiers 可配合長度指定敘述來達到輸出長度控制的目的。

【例如】

```
integer nth_pattern;  
reg a, b, c, d;  
$display("%8d: %b %2o %3h %4d", nth_pattern, a, b, c, d);
```

用 8 格的空間來輸出 `nth_pattern`，`a` 用 1 格，`b` 用 2 格，`c` 用 3 格，`d` 用 4 格。

資料長度如果比長度指定還要長，那麼仍會將資料（實數除外）原原本本的輸出。

資料長度如果比長度指定還要短，那麼會先輸出空白再輸出資料；對於八進制與十六進制值的輸出，則是先輸出空白接著 1 個領導 0 再輸出資料。

【例如：// display.TF】

```
reg [9:0] a3, b4, c5, d6;  
a3 = 10'h033;  
b4 = 10'h044;  
c5 = 10'h055;  
d6 = 10'h066;  
rd = 31.415;  
  
$display("a3 = %b, b4 = %o, c5 = %d, d6 = %h", a3, b4, c5, d6);  
輸出結果：a3 = 0000110011, b4 = 0104, c5 = 85, d6 = 066  
  
$display("a3 = %6b, b4 = %6o, c5 = %6d, d6 = %6h", a3, b4, c5, d6);  
輸出結果：a3 = 110011, b4 = 0104, c5 = 85, d6 = 066
```

`$displayb` 或 `$writeb` 會用 2 進制值來顯示。

【例如：// display.TF】

```
$displayb(a3,, b4,, c5,, d6);  
輸出結果：0000110011 0001000100 0001010101  
$writeb(a3,, b4,, c5,, d6);  
輸出結果：0000110011 0001000100 0001010101
```

\$display 或 \$write 如果沒有給定 format specifiers，那麼會用 10 進制值來顯示。

【例如：// display.TF】

```
$display(a3, b4, c5, d6);
輸出結果：51 68 85 102
$write(a3, b4, c5, d6);
輸出結果：51 68 85 102
```

\$displayh 或 \$writeh 會用 16 進制值來顯示。

【例如：// display.TF】

```
$displayh(a3,, b4,, c5,, d6);
輸出結果：033 044 055 066
$writeh(a3,, b4,, c5,, d6);
輸出結果：033 044 055 066
```

【例如】

```
reg [7:0] mem_byte, mem_reg[0:15];
mem_reg[2] = 8'b0101_1010; // 將資料放入到 mem_reg[2] 這個位址
$displayb(mem_reg[2]); // 用二進制格式輸出 mem_reg[2] 裡頭的值 (8 個位元)
輸出結果：01011010

mem_word = mem_reg[2]; // 取得 mem_reg[2] 裡頭的值 (8 個位元)
$displayb(mem_byte[6]); // 用二進制格式輸出 mem_byte[6] 這個位元
輸出結果：1
```

%e %E 使用指數的方式來顯示數值，%M.Ne 或 %M.NE，其中：N 表示小數部份所佔的位數，M 表示整個數值顯示所佔的位數 (包含小數部份)。

如果 M 給的太小，則指數顯示的部份會有誤差；如果 M 給的太大，則會顯示多的空格字元再顯示數值。如果 N 給的太小，則指數顯示的部份會有誤差；如果數值所佔的位數太長，則指數顯示的部份會有誤差，e+??? 算只佔一個空間。

`%f %F` 使用浮點數的方式來顯示數值，`%M.Nf` 或 `%M.NF`，其中：`N` 表示小數部份所佔的位數，`M` 表示整個數值顯示所佔的位數（包含小數部份）。

如果 `M` 給的太小，則小數顯示的部份會有誤差；如果 `M` 給的太大，則會顯示多的空格字元再顯示數值。如果 `N` 給的太小，則小數顯示的部份會有誤差；如果數值所佔的位數太長，則小數顯示的部份會有誤差，`e+???`算只佔一個空間。

`%g %G` 使用浮點數的方式來顯示數值，`%M.Ng` 或 `%M.NG`，其中：`N` 表示整個數值所佔的位數，`M` 表示整個數值顯示所佔的位數（包含小數部份）。

如果 `M` 給的太小，則小數顯示的部份會有誤差；如果 `M` 給的太大，則會顯示多的空格字元再顯示數值。如果 `N` 給的太小，則小數顯示的部份會有誤差；如果數值所佔的位數太長，則小數顯示的部份會有誤差，`e+???`算只佔一個空間。

【例如：`// display.TF`】

```
real rd;
rd = 314.1592;

$display("rd = %e, rd = %f, rd = %g", rd, rd, rd);
輸出結果：rd = 3.141592e+002, rd = 314.159200, rd = 314.159

$display("rd = %3.1e, rd = %10.1e, rd = %10.2e, rd = %10.3e, rd = %10.4e", rd, rd, rd, rd, rd);
輸出結果：rd = 3.1e+002, rd = 3.1e+002, rd = 3.14e+002, rd = 3.142e+002, rd = 3.1416e+002

$display("rd = %3.1f, rd = %10.1f, rd = %10.2f, rd = %10.3f, rd = %10.4f", rd, rd, rd, rd, rd);
輸出結果：rd = 314.2, rd = 314.2, rd = 314.16, rd = 314.159, rd = 314.1592

$display("rd = %3.1g, rd = %10.1g, rd = %10.2g, rd = %10.3g, rd = %10.4g", rd, rd, rd, rd, rd);
輸出結果：rd = 3e+002, rd = 3e+002, rd = 3.1e+002, rd = 314, rd = 314.2

$display("rd = %3.1E, rd = %10.1E, rd = %10.2F, rd = %10.3G, rd = %10.4G", rd, rd, rd, rd, rd);
輸出結果：rd = 3.1E+002, rd = 3.1E+002, rd = 314.16, rd = 314, rd = 314.2
```



14.3 Verilog 的除錯輔助功能

14.3.1 \$monitor 以及 \$fmonitor

\$monitor 與 \$fmonitor 基本上和 \$display 與 \$fdisplay 在語法與功能上是相同的。

不過 \$monitor 與 \$fmonitor 可以用 \$monitoron 來打開顯示輸出，可以用 \$monitoroff 來關閉顯示輸出。

在一個測試平台中，只能有 1 個 \$monitor 可以動作，\$fmonitor 可以有 1 個同時動作。

\$fopen 用於產生一個空的文字檔，配合 \$fdisplay、\$fwrite 以及 \$fstrobe 這一系列的 task，將電路模擬過程中產生的模擬結果或訊息輸出到檔案中。

\$time、\$stime 以及 \$realtime 這幾個 task 並不會觸動 \$monitor 或者是 \$fmonitor 這 2 個 task。

\$monitor 以及 \$fmonitor 這 2 個 task 的輸出與電路波型模擬的輸出 task 是互相獨立的，動作彼此沒有影響。

【語法】

```
$monitor( P1, P2, ... , Pn );  
$fmonitor( file_handle, P1, P2, ... , Pn );  
$monitoron;  
$monitoroff;
```

其中：file_handle 檔案代碼，請參考前面的 \$fopen 這個 task。參數 P1, P2, ... , Pn 可以是字串、變數、運算式或者是 ,, (表示) 這一欄的資料留空。字串可以：escaped characters 或者是 format specifiers。

【例如：// monitor.TF】

```
reg [3:0] a, b;

initial
begin : a_b_Apply
    integer i;
    a = 4'd0;
    for (i = 1; i <= 15; i = i + 1)
    begin
        #10
        a = a + 1;
    end
    $finish;
end

initial
begin
    $monitor("%t a=%d", $realtime, a);
    $fmonitor(Result, "%t a=%d", $realtime, a);
    $monitor("%t b=%d", $realtime, b);
    $fmonitor(Result, "%t b=%d", $realtime, b);
end

initial
begin : monitor_on_off
    $monitoroff;
    #40 // 40
    $monitoron;
    #40 // 80
    $monitoroff;
    #40 // 120
    $monitoron;
end
```

《\$monitor 輸出到畫面上的模擬結果》

```
0 b= 0
40 b= 4
50 b= 5
60 b= 6
70 b= 7
120 b=12
130 b=13
140 b=14
```

《\$fmonitor 輸出到檔案中的模擬結果》

```
0 a= 0
0 b= 1
40 a= 4
40 b= 5
50 a= 5
50 b= 6
60 a= 6
60 b= 7
70 a= 7
70 b= 8
120 a=12
120 b=13
130 a=13
130 b=14
140 a=14
140 b=15
```

14.3.2 \$strobe 以及 \$fstrobe

\$strobe 以及 \$fstrobe 用以在電路模擬單位時間的最後輸出穩定不再變化的值，用以得知單位時間的最後某些變數的值。

\$strobe 用於將模擬結果或訊息輸出到畫面上，\$fstrobe 則是用於將模擬結果或訊息輸出到檔案中。

\$strobe 和 \$display 的功能亦類似，不同在於 \$strobe 會輸出訊號在單位時間內穩定不再變化的值；而 \$display 則只要執行，不論訊號是否在單位時間穩定了都會輸出。

\$fstrobe 和 \$fdisplay 的功能亦類似，不同在於 \$fstrobe 會輸出訊號在單位時間內穩定不再變化的值；而 \$fdisplay 則只要執行，不論訊號是否在單位時間穩定了都會輸出。

【語法】

```
$strobe ( P1, P2, ... , Pn );  
$strobeb( P1, P2, ... , Pn );  
$strobeo( P1, P2, ... , Pn );  
$strobeh( P1, P2, ... , Pn );  
  
$fstrobe ( file_handle, P1, P2, ... , Pn );  
$fstrobeb( file_handle, P1, P2, ... , Pn );  
$fstrobeo( file_handle, P1, P2, ... , Pn );  
$fstrobeh( file_handle, P1, P2, ... , Pn );
```

其中：file_handle 檔案代碼，請參考前面的 \$fopen 這個 task。參數 P1, P2, ..., Pn 可以是字串、變數、運算式或者是,, (表示) 這一欄的資料留空。字串可以：escaped characters 或者是 format specifiers。

【例如】

```
initial  
begin  
  a = 5;  
  $display("a = ", a); // 輸出 a = 5  
  $strobe("a = ", a);  // 輸出 a = 10  
  a = 10;  
  
  #20  
  $finish;  
End
```

從這個例子中可以發現同樣是在時間 0，\$display 輸出 a 的值是 5，而 \$strobe 輸出 a 的值卻是 10。



14.4 Verilog 的時間格式與精確度

14.4.1 \$timeformat

\$timeformat 用來定義 \$display 及 \$monitor... 等等可以輸出時間的 task，它們在電路模擬的時間單位。

`timescale 與 \$timeformat 會影響到 \$display 及 \$monitorand，用 %t 這個 format specifier 來顯示目前電路的模擬時間。

【語法】

```
$timeformat[( Units, Precision, Suffix, MinFieldWidth)];
```

其中：

Units	Precision
0	1 s (s: second)
-1	100 ms (ms: millisecond)
-2	10 ms
-3	1 ms
-4	100 us (us: microsecond)
-5	10 us
-6	1 us
-7	100 ns (ns: nanosecond)
-8	10 ns
-9	1 ns
-10	100 ps (ps: picosecond)
-11	10 ps
-12	1 ps
-13	100 fs (fs: femtosecond)
-14	10 fs
-15	1 fs

Precision 表示時間值小數的部份所佔的位數。

Suffix 必須是字串，會輸出在時間值的後面。

MinFieldWidth 表示要用多少個字元空間來顯示時間值 (包括長度太長先顯示的空白)，如果 MinFieldWidth 的值給的不夠大，那麼時間值是多少就照示，也就是忽略 MinFieldWidth 不管。

預設是：Units 等於 `timescale 模擬時間單位/精確度中的模擬時間單位，Precision 等於 0，Suffix 等於空字串，MinFieldWidth 等於 20 個字元。

【例如：// timeformat.TF】

```
`timescale 100 ps / 10 ps
//      time unit / resolution

module t;
  reg [2:0] a;

  initial
    $timeformat(-9, 3, " x 100 ps", 20); // 20.3 x 100 ps

  initial
  begin : a_Apply
    integer i;
    a = 3'd0;
    for (i = 1; i <= 7; i = i + 1)
      begin
        #10 a = a + 1;
      end
      #10 $finish(1);
    end

  initial
    $monitor("%t: a = %d", $time, a);
endmodule
```

輸出結果：

```
0.000 x 100 ps: a = 0
1.000 x 100 ps: a = 1
2.000 x 100 ps: a = 2
3.000 x 100 ps: a = 3
4.000 x 100 ps: a = 4
5.000 x 100 ps: a = 5
6.000 x 100 ps: a = 6
7.000 x 100 ps: a = 7
```

【例如：// timeformat2.TF】

```
`timescale 10 ns / 1 ns

module t;
  reg in1;

  initial
    $timeformat(-9, 5, " ns", 10); // 10.5 ns

  not #1 (out1, in1);

  initial
    $monitor("%m: %t, in1 = %d, out1 = %h", $realtime, in1, out1);

  initial
  begin
    in1 = 0;
    #10 in1 = 1;
    #10 in1 = 0;

    #10 $finish;
  end
endmodule
```

輸出結果：

```
t: 0.00000 ns, in1 = 0, out1 = x
t: 10.00000 ns, in1 = 0, out1 = 1
t: 100.00000 ns, in1 = 1, out1 = 1
t: 110.00000 ns, in1 = 1, out1 = 0
t: 200.00000 ns, in1 = 0, out1 = 0
t: 210.00000 ns, in1 = 0, out1 = 1
```

14.4.2 \$printtimescale

`printtimescale` 用於顯示某個模組 (module) 在模擬時所用的電路模擬時間單位與精確度，會以 `Time scale of (階層式電路格式)` 電路模擬時間單位／精確度的表示表來顯示。

如果該模組 (module) 的 .v 檔案中沒有指定 ``timescale`，會以電路最上層的 .v 檔案中指定的 ``timescale` 來當作這個模組 (module) 的 ``timescale`。

【語法】

```
$sprnttimescale[ (ModuleInstanceName) ];
```

【例如】

```
// module_a.v
`timescale 1 ms / 100 us

module module_a(a, b);
input  [3:0] a;
output [3:0] b;
reg     [3:0] b;

  module_b inst_b();

  initial
  begin
    $sprnttimescale;
    $sprnttimescale(inst_b);
    $sprnttimescale(inst_b.inst_c);
  end

  always @( a )
  begin
    b = a + 1;
  end

endmodule

// module_b.v
`timescale 1 us / 100 ns

module module_b;

  module_c inst_c();

  initial
  begin
    $sprnttimescale;
    $sprnttimescale(inst_c);
  end

endmodule
```

```
// module_v.v
`timescale 10 ns / 1 ns

module module_c;

    initial
        $printtimescale;

endmodule

// printtimescale.TF
`timescale 1ns / 1ns

module t;
    reg [3:0] a;
    wire [3:0] b;
    ...

    module_a inst_a (.a(a), .b(b) );

    initial
    begin
        $printtimescale;
        $printtimescale(inst_a);
        $printtimescale(inst_a.inst_b);
        $printtimescale(inst_a.inst_b.inst_c);
    end

endmodule
```

輸出結果：

```
Time scale of (t.inst_a.inst_b.inst_c) is 10ns / 1ns
Time scale of (t.inst_a.inst_b) is 1us / 100ns
Time scale of (t.inst_a.inst_b.inst_c) is 10ns / 1ns
Time scale of (t.inst_a) is 1ms / 100us
Time scale of (t.inst_a.inst_b) is 1us / 100ns
Time scale of (t.inst_a.inst_b.inst_c) is 10ns / 1ns
Time scale of (t) is 1ns / 1ns
Time scale of (t.inst_a) is 1ms / 100us
Time scale of (t.inst_a.inst_b) is 1us / 100ns
Time scale of (t.inst_a.inst_b.inst_c) is 10ns / 1ns
```

試著將 module_a.v 的 `//`timescale 1 ms / 100 us` 前面加上 `//`，以及在 module\_c.v 的 `//`timescale 10 ns / 1 ns` 前面加上 `//`，那麼輸出結果：

```

Time scale of (t.inst_a.inst_b.inst_c) is 1ns / 1ns
Time scale of (t.inst_a.inst_b) is 1us / 100ns
Time scale of (t.inst_a.inst_b.inst_c) is 1ns / 1ns
Time scale of (t.inst_a) is 1ns / 1ns
Time scale of (t.inst_a.inst_b) is 1us / 100ns
Time scale of (t.inst_a.inst_b.inst_c) is 1ns / 1ns
Time scale of (t) is 1ns / 1ns
Time scale of (t.inst_a) is 1ns / 1ns
Time scale of (t.inst_a.inst_b) is 1us / 100ns
Time scale of (t.inst_a.inst_b.inst_c) is 1ns / 1ns

```

14.4.3 \$time、\$stime 以及 \$realtime

\$time、\$stime 以及 \$realtime 都可以用來取得目前電路模擬的時間。

傳回時間值單位的是 `timescale 模擬時間單位／精確度中的模擬時間單位。

\$time 傳回 64 個位元的無號數整數，四捨五入到最接近模擬時間單位為原則。

\$stime 傳回 32 個位元的無號數整數，如果值太大會被截掉。

\$realtime 傳回實數的時間值。

【例如：// time.TF】

```

`timescale 1 ns / 10 ps
module tb;
  parameter p = 2.46;
  reg value;
  Initial
  begin
    $monitor($time,, "value = ", value);
  或
    $monitor($stime,, "value = ", value);
    #p value = 0;
    #p value = 1;
  end
endmodule

```

輸出結果：

```
0 value = x
2 value = 0
5 value = 1
```

【例如】

```
`timescale 1 ns / 10 ps
module tb;
  parameter p = 2.46;
  reg value;
  initial
  begin
    $monitor($realtime,, "value = ", value);
    #p value = 0;
    #p value = 1;
  end
endmodule
```

輸出結果：

```
0 value = x
2.46 value = 0
4.92 value = 1
```



14.5 資料型態轉換

14.5.1 \$realtobits 以及 \$bitstoreal

\$realtobits 輸入的參數是實數，傳回的是 64 個位元值。

\$bitstoreal 輸入的參數是 64 個位元值，傳回的是實數。

【語法】

```
BitValue = $realtobits( RealExpression );
RealValue = $bitstoreal( BitValueExpression );
```

【例如：// real_bits.TF】

```
module tb;
  reg [64:1] reg_value1;
  real      real_value1;

  reg [64:1] reg_value2;
  real      real_value2;

  initial
  begin
    real_value1 = 3.14;
    reg_value1 = $realtobits(real_value1);
    $display("real_value1 = %f, reg_value1 = %d", real_value1,
      reg_value1);
  end

  initial
  begin
    reg_value2 = 64'd4614253070214989087;
    real_value2 = $bitstoreal(reg_value2);
    $display("reg_value2 = %d, real_value2 = %f", reg_value2,
      real_value2);
  end
endmodule
```

輸出結果：

```
real_value1 = 3.140000, reg_value1 = 4614253070214989087
reg_value2 = 4614253070214989087, real_value2 = 3.140000
```

14.5.2 \$rtol 以及 \$itor

\$rtol 將實數轉換成整數，小數的部份會截斷。例如：123.45 轉換後變成 123。

\$itor 將整數轉換成實數。例如：123 轉換後變成 123.0。

【語法】

```
$rtol( RealExpression )
$itor( IntegerExpression )
```


【例如：/ ri_ir.TF】

```
module tb;
  real    real_value1;
  integer int_value1;

  integer int_value2;
  real    real_value2;

  initial
  begin
    real_value1 = 3.14;
    int_value1 = $rtol(real_value1);
    $display("real_value1 = %f, int_value1 = %d", real_value1,
    int_value1);
  end

  initial
  begin
    int_value2 = 1234;
    real_value2 = $itor(int_value2);
    $display("int_value2 = %d, real_value2 = %f", int_value2,
    real_value2);
  end
endmodule
```

輸出結果：

```
real_value1 = 3.140000, int_value1 =          3
int_value2 =          1234, real_value2 = 1234.000000
```



14.6 Verilog 的系統任務

\$stop 用於暫停電路的模擬。

【語法】

```
$stop;
```

\$finish 用於中止電路的模擬。

【語法】

```
$finish(n); // n 可以是 0, 1, 或 2
```

其中：(n) 是可有可無的，預設值是 (1)，也就是說，寫成 \$finish 與 \$finish(1) 是相同的。

(0) 表示不輸出訊息。

(1) 表示要輸出電路的模擬時間與停止時電路正模擬到的電路位置。

```
** Note: $finish      : timeformat.TF(18)
      Time: 8 ns  Iteration: 0  Instance: /t
```

(2) 表示要輸出電路的模擬時間與停止時電路正模擬到的電路位置、系統記憶體與 CPU 使用的狀況。

```
** Note: Data structure takes 1310736 bytes of memory
      Process time 0.00 seconds
      $finish      : timeformat.TF(18)
      Time: 8 ns  Iteration: 0  Instance: /t
```

本章練習

Question1 :

什麼時候會用到 Verilog 的檔案處理 ?

Question2 :

什麼時候會用到 Verilog 的除錯輔助 ?

Question3 :

什麼時候會用到 Verilog 的時間格式 ?

Question4 :

什麼時候需要資料型態轉換 ?

PS Google 可以幫您找到解答。